

Day 6: Implementing PIDs

-Tune PD first, then tune I once steady state residual is only thing left

-4 metrics for theta response:

-Rise time: how fast theta reaches target; mostly changed by K_p . Lower is better until it costs overshoot

-Overshoot: driven down by K_d

-Settling Time: how long until it stops oscillating to an acceptable level

-Steady State error: error after settling. Should go down to zero with K_i

-Must trade between the 4, no perfect sol. Name trade-offs explicitly when explaining weighting.

3 common failure modes:

-Too much P: sustained oscillation

-Constantly overshooting and having to correct itself

-Too much D: Noise gets amplified, making it super jittery

-Too much I: Windup. Too much correction is stored, causing large secondary oscillation. Fixed with noting actuator saturation.

Implement today in ideal conditions to use today's tuning table as a control to find out which non-ideal condition is causing problems.

Matlab Understanding PID Control

Video 2: Anti-windup

- Risk of actuator saturation if we constantly let I term build
- Issue isn't saturation per se, but instead that we have to unwind the excessive I term afterwards, disrupting control logic.
- Therefore we will try to keep int. value within some limit. 3 ways:
 - Clamping
 - Check if saturated
 - Check sign of output and sign of error
 - If both are true, controller is contributing to sat. and we need to turn off integrator (set input to 0 so no more adding)
 - Back-Calculation
 - Proportionally feed back difference between commanded and actual actuator output to "unwind" integrator. Requires another gain
 - Observer Approach

Video 3: Noise Filtering

- Derivative term is massively affected by high frequency noise
- Use low pass filter, trick is determining where the cutoff is

$$\frac{N}{S+N}$$

In Laplace domain, low pass filter represented by

Where N is cutoff frequency in rad/s

Lesson 4: Tuning

- Start with definite requirements
 - time domain (rise time, etc.)
 - frequency domain (bandwidth, damping ratio)
- Understand system (stable? Linear?)
- Choose controller
- Choose gains
- Validation

PID Tuning Guide

- Marks of poorly behaved system: non-linear, unstable, delay
 - More of these are true, less likely the methods shown will work
- Can use a model or prototype
- Ziegler-Nichols technique is one possible method
- Gives only initial guess
- Models
 - From first principles (like EOM model)
 - System ID: generate specific transfer function based on tests (will use this

for servos)

-Then choose gains based on control theory (e.g. pole placement)

-Or use heuristics (like Ziegler-Nichols)

Day 7 Content

3 Big Servo Imperfections:

-Saturation: limited gimbal angle in our case. Forces *open loop*

-Rate Limit: can only move so fast

-Lag/transport delay. Another side effect when combined with 2 is that rapid reversals will get rounded off.

Flight Computer operates discretely, model using zero-order holds in Simulink

-What is real difference between point 1 and point 2 in website?